

MongoDB Schemas — 3D Forge E-Commerce Platform

Version: 1.0

Date: June 30, 2026

Stack: Node.js + Express + Mongoose + MongoDB Atlas

Reference PRD: 3D Forge PRD v1.0

All schemas use Mongoose. Each model file is listed in the order it should be created (dependencies first). Every schema includes timestamps (createdAt, updatedAt) via { timestamps: true }.

Schema Dependency Order

1. User
2. Category
3. Product
4. Cart
5. Coupon
6. CustomOrderRequest
7. Order
8. Review
9. Wishlist
10. Notification

1. User Schema

Used by: Auth (/api/auth/*), Orders, Reviews, Wishlist, Cart, Custom Orders

```
// models/User.js
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');

const addressSchema = new mongoose.Schema({
  label:      { type: String, default: 'Home' },           // 'Home', 'Work', 'Other'
  fullName:   { type: String, required: true },
  phone:      { type: String, required: true },
  line1:      { type: String, required: true },
  line2:      { type: String },
  city:       { type: String, required: true },
  state:      { type: String, required: true },
  pincode:    { type: String, required: true },
  isDefault:  { type: Boolean, default: false },
}, { _id: true });

const userSchema = new mongoose.Schema({
  name:       { type: String, required: true, trim: true },
```

```

    email:      { type: String, required: true, unique: true, lowercase: true, trim: true },
    passwordHash: { type: String }, // null if OAuth-only
    phone:      { type: String },
    role:       { type: String, enum: ['customer', 'admin', 'super_admin'], default: 'customer' },
    isEmailVerified: { type: Boolean, default: false },
    isBlocked:    { type: Boolean, default: false },
    oauthProvider: { type: String, enum: ['google', null], default: null },
    oauthId:      { type: String },
    addresses:    [addressSchema],
    emailVerifyToken: { type: String },
    emailVerifyExpires: { type: Date },
    passwordResetToken: { type: String },
    passwordResetExpires: { type: Date },
    refreshToken: { type: String }, // hashed refresh token
  }, { timestamps: true });

// Hash password before save
userSchema.pre('save', async function (next) {
  if (!this.isModified('passwordHash')) return next();
  this.passwordHash = await bcrypt.hash(this.passwordHash, 12);
  next();
});

userSchema.methods.comparePassword = function (plain) {
  return bcrypt.compare(plain, this.passwordHash);
};

// Never return sensitive fields
userSchema.methods.toSafeObject = function () {
  const { passwordHash, refreshToken, emailVerifyToken, passwordResetToken, ...safe } = this.toObject();
  return safe;
};

// Indexes
userSchema.index({ email: 1 });
userSchema.index({ role: 1 });

module.exports = mongoose.model('User', userSchema);

```

API Mapping

Endpoint	Schema Fields Used
POST /api/auth/register	name,email,passwordHash,emailVerifyToken, emailVerifyExpires
POST /api/auth/login	email,passwordHash,isBlocked,isEmailVerified, refreshToken
POST /api/auth/logout	refreshToken (cleared)
POST /api/auth/refresh	refreshToken
POST /api/auth/forgot-password	passwordResetToken,passwordResetExpires

Endpoint	Schema Fields Used
POST /api/auth/reset-password	passwordHash,passwordResetToken,passwordResetExpires
GET /api/users/:id(<i>Admin</i>)	full document
PUT /api/users/addresses	addresses[]

2. Category Schema

Used by: Products, Coupons, Admin Dashboard

```
// models/Category.js
const mongoose = require('mongoose');

const categorySchema = new mongoose.Schema({
  name:      { type: String, required: true, unique: true, trim: true },
  slug:      { type: String, required: true, unique: true, lowercase: true },
  description: { type: String },
  image:      { type: String },                                // CDN URL
  isActive:   { type: Boolean, default: true },
  sortOrder:  { type: Number, default: 0 },
}, { timestamps: true });

categorySchema.index({ slug: 1 });

module.exports = mongoose.model('Category', categorySchema);
```

API Mapping

Endpoint	Schema Fields Used
GET /api/categories	all active categories
POST /api/categories(<i>Admin</i>)	name,slug,description,image
PUT /api/categories/:id(<i>Admin</i>)	any field
DELETE /api/categories/:id(<i>Admin</i>)	soft delete via isActive: false

3. Product Schema

Used by: Product catalog, Cart, Orders, Reviews, Wishlist, Search

```
// models/Product.js
const mongoose = require('mongoose');

const variantSchema = new mongoose.Schema({
  label:    { type: String, required: true },    // e.g., "Small - Black - PLA"
  material: { type: String },                    // e.g., "PLA", "Resin", "PETG"
  color:    { type: String },
}
```

```

    size:      { type: String },          // e.g., "10x10x5 cm"
    price:     { type: Number, required: true, min: 0 },
    stock:     { type: Number, default: 0, min: 0 },
    sku:       { type: String },
  }, { _id: true });

const dimensionSchema = new mongoose.Schema({
  length: Number,
  width: Number,
  height: Number,
  unit:   { type: String, default: 'cm' },
  weight: { type: Number },              // in grams
}, { _id: false });

const productSchema = new mongoose.Schema({
  title:      { type: String, required: true, trim: true },
  slug:       { type: String, required: true, unique: true, lowercase: true },
  description: { type: String, required: true },
  shortDesc:  { type: String },          // for product cards (max 160 chars)
  category:   { type: mongoose.Schema.Types.ObjectId, ref: 'Category', required: true },
  tags:       [{ type: String, lowercase: true }],
  images:     [{ type: String }],        // CDN URLs, first = primary
  variants:   [variantSchema],
  dimensions: dimensionSchema,
  printTime:  { type: String },          // e.g., "4-6 hours"
  isCustomizable: { type: Boolean, default: false },
  isActive:   { type: Boolean, default: true },
  isFeatured: { type: Boolean, default: false },
  averageRating: { type: Number, default: 0, min: 0, max: 5 },
  reviewCount: { type: Number, default: 0 },
  totalSold:  { type: Number, default: 0 },
  metaTitle:  { type: String },          // SEO
  metaDescription: { type: String },      // SEO
}, { timestamps: true });

// Full-text search index
productSchema.index({ title: 'text', description: 'text', tags: 'text' });
productSchema.index({ slug: 1 });
productSchema.index({ category: 1 });
productSchema.index({ isActive: 1, isFeatured: 1 });
productSchema.index({ averageRating: -1 });
productSchema.index({ totalSold: -1 });

module.exports = mongoose.model('Product', productSchema);

```

API Mapping

Endpoint	Schema Fields Used
GET /api/products	title,slug,images[0],variants[0].price,averageRating,reviewCount,isActive,category,tags
GET /api/products/:slug	full document (populated category)
POST /api/products (Admin)	all fields

Endpoint	Schema Fields Used
PUT /api/products/:id (Admin)	any field
DELETE /api/products/:id (Admin)	isActive: false (soft delete)

Query Params for GET /api/products

Param	Filters on
q	Full-text search (\$text)
category	category (ObjectId)
minPrice/maxPrice	variants.price
material	variants.material
rating	averageRating
inStock	variants.stock > 0
sort	createdAt,price,averageRating,totalSold
page/limit	pagination

4. Cart Schema

Used by: Cart (/api/cart/*), Checkout

```
// models/Cart.js
const mongoose = require('mongoose');

const cartItemSchema = new mongoose.Schema({
  product: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true },
  variantId: { type: mongoose.Schema.Types.ObjectId, required: true },
  quantity: { type: Number, required: true, min: 1, default: 1 },
  // Snapshot at time of add (to show "price changed" alerts)
  priceAtAdd: { type: Number, required: true },
}, { _id: true });

const cartSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  items: [cartItemSchema],
  couponApplied: {
    code: { type: String },
    discountAmount: { type: Number, default: 0 },
  },
}, { timestamps: true });

cartSchema.index({ user: 1 });
```

```
module.exports = mongoose.model('Cart', cartSchema);
```

API Mapping

Endpoint	Schema Fields Used
GET /api/cart	full doc, populated product
POST /api/cart/add	items[] push
PUT /api/cart/update	items[].quantity
DELETE /api/cart/remove	items[] pull by _id
POST /api/cart/apply-coupon	couponApplied
DELETE /api/cart/remove-coupon	couponApplied cleared

5. Coupon Schema

Used by: Cart, Orders, Admin Dashboard (/api/coupons/*)

```
// models/Coupon.js
const mongoose = require('mongoose');

const couponSchema = new mongoose.Schema({
  code: { type: String, required: true, unique: true, uppercase: true, trim: true },
  description: { type: String },
  discountType: { type: String, enum: ['flat', 'percentage'], required: true },
  discountValue: { type: Number, required: true, min: 0 }, // ₹ or %
  maxDiscount: { type: Number }, // cap for percentage
  minOrderValue: { type: Number, default: 0 },
  expiresAt: { type: Date, required: true },
  usageLimit: { type: Number, default: null }, // null = unlimited
  usageLimitPerUser: { type: Number, default: 1 },
  usedCount: { type: Number, default: 0 },
  usedBy: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
  applicableTo: {
    type: { type: String, enum: ['all', 'category', 'product'], default: 'all' },
    categories: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Category' }],
    products: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Product' }],
  },
  isActive: { type: Boolean, default: true },
}, { timestamps: true });

couponSchema.index({ code: 1 });
couponSchema.index({ expiresAt: 1 });

// Instance method: calculate discount for a given order subtotal
couponSchema.methods.calculateDiscount = function (subtotal) {
  if (this.discountType === 'flat') return Math.min(this.discountValue, subtotal);
  const pct = (subtotal * this.discountValue) / 100;
  return this.maxDiscount ? Math.min(pct, this.maxDiscount) : pct;
};
```

```
};

module.exports = mongoose.model('Coupon', couponSchema);
```

API Mapping

Endpoint	Schema Fields Used
POST /api/coupons/validate	code, isActive, expiresAt, minOrderValue, usageLimit, usedCount, usageLimitPerUser, usedBy
POST /api/coupons (Admin)	all fields
GET /api/coupons (Admin)	all (with usedCount, usedBy stats)
PUT /api/coupons/:id (Admin)	any field
DELETE /api/coupons/:id (Admin)	isActive: false

6. CustomOrderRequest Schema

Used by: Custom Order flow (/api/custom-orders/*)

```
// models/CustomOrderRequest.js
const mongoose = require('mongoose');

const timelineEventSchema = new mongoose.Schema({
  status: { type: String, required: true },
  note: { type: String },
  updatedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  at: { type: Date, default: Date.now },
}, { _id: false });

const customOrderRequestSchema = new mongoose.Schema({
  requestId: { type: String, unique: true }, // e.g., "COR-2026-0001"
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  // Step 1 – Idea
  description: { type: String, required: true },
  referenceImages: [{ type: String }], // CDN URLs (Firebase Storage)
  preferredMaterial: { type: String },
  preferredColor: { type: String },
  // Step 2 – Dimensions
  dimensions: {
    length: { type: Number },
    width: { type: Number },
    height: { type: Number },
    unit: { type: String, default: 'cm' },
  },
  quantity: { type: Number, required: true, min: 1, default: 1 },
  deadlinePreference: { type: String }, // "1 week", "2 weeks"
  // Step 3 – Contact
  budgetRange: {
```

```

    min: { type: Number },
    max: { type: Number },
  },
  preferredContact: { type: String, enum: ['whatsapp', 'email'], default: 'email' },
  additionalNotes: { type: String },
  // Admin fields
  status: {
    type: String,
    enum: ['pending_review', 'quoted', 'accepted', 'rejected', 'in_production', 'shipping'],
    default: 'pending_review',
  },
  quote: {
    amount: { type: Number },
    estimatedDays: { type: Number },
    adminNote: { type: String },
    quotedAt: { type: Date },
    quotedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  },
  convertedOrderId: { type: mongoose.Schema.Types.ObjectId, ref: 'Order' }, // after
  timeline: [timelineEventSchema],
}, { timestamps: true });

// Auto-generate requestId before save
customOrderRequestSchema.pre('save', async function (next) {
  if (!this.requestId) {
    const count = await mongoose.model('CustomOrderRequest').countDocuments();
    this.requestId = `COR-${new Date().getFullYear()}-${String(count + 1).padStart(5, '0')}`;
  }
  next();
});

customOrderRequestSchema.index({ user: 1 });
customOrderRequestSchema.index({ status: 1 });
customOrderRequestSchema.index({ requestId: 1 });

module.exports = mongoose.model('CustomOrderRequest', customOrderRequestSchema);

```

API Mapping

Endpoint	Schema Fields Used
POST /api/custom-orders	all Step 1-3 fields, user, status: 'pending_review'
GET /api/custom-orders (Admin)	full doc, populated user
GET /api/custom-orders/:id	full doc
PUT /api/custom-orders/:id/status (Admin)	status, quote, timeline push
PUT /api/custom-orders/:id/accept-quote (Customer)	status: 'accepted'
PUT /api/custom-orders/:id/reject-quote (Customer)	status: 'rejected'

7. Order Schema

Used by: Checkout, Order tracking, Admin Dashboard, Razorpay Webhook (/api/orders/*, /api/payments/*)

[illegible]

```

// Fulfillment
orderStatus: {
  type: String,
  enum: [
    'pending',
    'payment_confirmed',
    'in_production',
    'quality_check',
    'shipped',
    'delivered',
    'cancelled',
    'refund_initiated',
    'refunded',
  ],
  default: 'pending',
},
trackingNumber: { type: String },
logisticsPartner: { type: String }, // "Shiprocket", "Delhivery", etc.
logisticsTrackingUrl: { type: String },
estimatedDelivery: { type: Date },
deliveredAt: { type: Date },
cancelReason: { type: String },
// Custom order link
customOrderRequest: { type: mongoose.Schema.Types.ObjectId, ref: 'CustomOrderRequest' },
timeline: [timelineEventSchema],
invoiceUrl: { type: String }, // PDF CDN URL
}, { timestamps: true });

// Auto-generate orderId
orderSchema.pre('save', async function (next) {
  if (!this.orderId) {
    const count = await mongoose.model('Order').countDocuments();
    this.orderId = `ORD-${new Date().getFullYear()}-${String(count + 1).padStart(5, '0')}`;
  }
  next();
});

orderSchema.index({ user: 1 });
orderSchema.index({ orderId: 1 });
orderSchema.index({ orderStatus: 1 });
orderSchema.index({ paymentStatus: 1 });
orderSchema.index({ razorpayOrderId: 1 });
orderSchema.index({ createdAt: -1 });

module.exports = mongoose.model('Order', orderSchema);

```

API Mapping

Endpoint	Schema Fields Used
POST /api/orders	user, items[], shippingAddress, subtotal, discount, shippingCost, total, coupon, paymentMethod
GET /api/orders (<i>Admin</i>)	full doc, filter by orderStatus, paymentStatus, date range

Endpoint	Schema Fields Used
GET /api/orders/my (<i>Customer</i>)	filter by user
GET /api/orders/:id	full doc (populated user, items.product)
PUT /api/orders/:id/status (<i>Admin</i>)	orderStatus, trackingNumber, logisticsPartner, timeline.push
POST /api/orders/:id/cancel (<i>Customer</i>)	orderStatus: 'cancelled', cancelReason
POST /api/payments/create-order	creates Razorpay order, stores razorpayOrderId
POST /api/payments/verify	razorpayPaymentId, razorpaySignature, paymentStatus: 'paid', orderStatus: 'payment_confirmed'
POST /api/payments/webhook	paymentStatus, orderStatus update via Razorpay event

8. Review Schema

Used by: Reviews (/api/reviews/*), Product averageRating and reviewCount updates

```
// models/Review.js
const mongoose = require('mongoose');

const reviewSchema = new mongoose.Schema({
  product:      { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true },
  user:         { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  order:        { type: mongoose.Schema.Types.ObjectId, ref: 'Order', required: true },
  rating:       { type: Number, required: true, min: 1, max: 5 },
  title:        { type: String, maxlength: 100 },
  comment:      { type: String, maxlength: 1000 },
  images:       [{ type: String }], // optional review images (CDN links)
  isVerifiedPurchase: { type: Boolean, default: true },
  isHidden:     { type: Boolean, default: false },
  helpfulVotes: { type: Number, default: 0 },
  helpfulVotedBy: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
}, { timestamps: true });

// One review per user per product
reviewSchema.index({ product: 1, user: 1 }, { unique: true });
reviewSchema.index({ product: 1, rating: -1 });
reviewSchema.index({ isHidden: 1 });

// After save/delete: recalculate product averageRating and reviewCount
reviewSchema.post('save', async function () {
  const stats = await mongoose.model('Review').aggregate([
    { $match: { product: this.product, isHidden: false } },
    { $group: { _id: '$product', avgRating: { $avg: '$rating' }, count: { $sum: 1 } } }
  ]);
  await mongoose.model('Product').findByIdAndUpdate(this.product, {
    averageRating: stats[0]?.avgRating?.toFixed(1) || 0,
    reviewCount:   stats[0]?.count || 0,
  });
});
```

```

    });
  });

  module.exports = mongoose.model('Review', reviewSchema);

```

API Mapping

Endpoint	Schema Fields Used
POST /api/reviews	product,user,order,rating,title,comment,images
GET /api/reviews/product/:productId	filter product,isHidden: false; sort options
PUT /api/reviews/:id	rating,title,comment (own review only)
DELETE /api/reviews/:id (Admin)	isHidden: true
POST /api/reviews/:id/helpful	helpfulVotes,helpfulVotedBy

9. Wishlist Schema

Used by: Wishlist (/api/wishlist/*)

```

// models/Wishlist.js
const mongoose = require('mongoose');

const wishlistSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, unique: true },
  products: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Product' }],
}, { timestamps: true });

wishlistSchema.index({ user: 1 });

module.exports = mongoose.model('Wishlist', wishlistSchema);

```

API Mapping

Endpoint	Schema Fields Used
GET /api/wishlist	full doc, populated products
POST /api/wishlist/add	\$addToSet: { products: productId }
DELETE /api/wishlist/remove/:productId	\$pull: { products: productId }

10. Notification Schema

Used by: In-app notifications for order updates, custom order quotes, coupon alerts

```
// models/Notification.js
const mongoose = require('mongoose');

const notificationSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  type: {
    type: String,
    enum: [
      'order_placed',
      'payment_confirmed',
      'order_shipped',
      'order_delivered',
      'custom_order_quoted',
      'custom_order_status',
      'coupon_available',
      'review_requested',
      'price_drop',
    ],
    required: true,
  },
  title: { type: String, required: true },
  message: { type: String, required: true },
  link: { type: String }, // e.g., '/orders/ORD-2026-00189'
  isRead: { type: Boolean, default: false },
  meta: { type: mongoose.Schema.Types.Mixed }, // extra context (orderId, request
}, { timestamps: true });

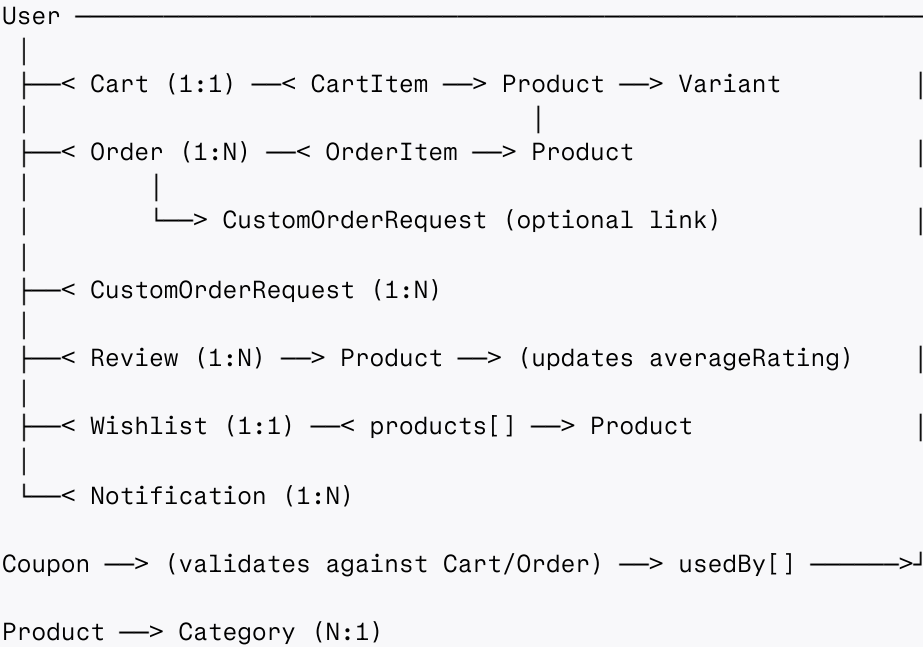
notificationSchema.index({ user: 1, isRead: 1 });
notificationSchema.index({ createdAt: -1 });

module.exports = mongoose.model('Notification', notificationSchema);
```

API Mapping

Endpoint	Schema Fields Used
GET /api/notifications	filter by user, isRead
PUT /api/notifications/:id/read	isRead: true
PUT /api/notifications/read-all	isRead: true for all user's notifications
DELETE /api/notifications/:id	delete document

Complete Schema Relationship Map



Index Summary

Model	Index	Type
User	email	Unique
User	role	Regular
Product	title, description, tags	Text (full-text search)
Product	slug	Unique
Product	category,isActive,isFeatured	Compound
Product	averageRating,totalSold	Descending
Order	user,orderId,orderStatus	Regular
Order	razorpayOrderId	Regular
Order	createdAt	Descending
Review	product + user	Unique Compound
Coupon	code,expiresAt	Regular
CustomOrderRequest	requestId,user,status	Regular
Notification	user + isRead,createdAt	Compound

Environment Variables Required

```
# Server
PORT=5000
NODE_ENV=development

# MongoDB
MONGO_URI=mongodb+srv://<user>:<pass>@cluster.mongodb.net/3dforge

# JWT
JWT_ACCESS_SECRET=<secret>
JWT_REFRESH_SECRET=<secret>
JWT_ACCESS_EXPIRES=15m
JWT_REFRESH_EXPIRES=7d

# Razorpay
RAZORPAY_KEY_ID=rzp_live_xxx
RAZORPAY_KEY_SECRET=xxx
RAZORPAY_WEBHOOK_SECRET=xxx

# Firebase (optional)
FIREBASE_PROJECT_ID=xxx
FIREBASE_STORAGE_BUCKET=xxx.appspot.com
FIREBASE_SERVICE_ACCOUNT_KEY=<base64 encoded JSON>

# Email (Nodemailer)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your@email.com
SMTP_PASS=app_password

# Cloudinary (alternative to Firebase Storage)
CLOUDINARY_CLOUD_NAME=xxx
CLOUDINARY_API_KEY=xxx
CLOUDINARY_API_SECRET=xxx

# Frontend
CLIENT_URL=https://yourdomain.com
```

End of Document — MongoDB Schemas v1.0